

Documenting progress

Straight from the VCAA Applied Computing Study Design (Unit 2, Area of Study 1), the examinable wording is:

What this key knowledge covers

This summary distils everything you need for **KK09 — Documenting progress** in Innovative Solutions. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

What this dot point actually asks of you

Straight from the VCAA Applied Computing Study Design (Unit 2, Area of Study 1), the examinable wording is:

“techniques for documenting the development of solutions, including the tools and techniques for **coordinating and monitoring projects, such as Gantt charts.**”

That one sentence hides three jobs you must be able to do and explain. Across this module you will master each, see them on Jade's real team build, and practise the exact wording examiners reward.

Use a **Gantt chart** to plan tasks, sequence them, set durations, milestones and dependencies — then track slippage.

Keep a **project journal / log** of decisions, ideas, problems and modifications as they happen.

Use **version control** to save dated copies so you can roll back and prove the work is your own.

FloatMate is a wearable wristband that buzzes a swimmer when they drift into a rip current. Year 11 student **Jade** — a Nippers volunteer — leads a four-person team building it across a 10-week term. From here on, every example is their build. Documentation is what stops four enthusiastic people from quietly building four different projects.

Why document at all? Four reasons that matter in the exam

“Because the teacher said so” earns zero marks. Examiners want the *purpose*. Documentation exists to solve four real problems on a team project.

□ Delegation

A clear plan lets the team split work so no two people unknowingly build the same thing — and nobody sits idle. Jade can assign the buzzer code to Sam while Priya designs the 3D enclosure, *because the Gantt chart shows those tasks don't depend on each other.*

□ Monitoring

Plans drift. A living Gantt chart and journal let the team see *this week* whether they're ahead, on time, or sliding — early enough to act, not the night before submission.

□ Recovery

When today's code breaks, version control lets you return to Friday's working copy in seconds instead of rebuilding from memory. Documentation is a safety net.

□ Authentication

A dated journal and version history *prove* the solution grew over time and is genuinely the team's own work — the basis of academic integrity in a SAT/SAC.

Think of building FloatMate as a four-car road trip. The **Gantt chart** is the route map and timetable — who drives which leg, by when. The **project journal** is the travel diary — where you stopped, what went wrong, the detour you chose and why. **Version control** is the set of save-points: if you take a wrong turn, you can drive back to the last good fork instead of restarting in Melbourne.

Three techniques — and the one-line difference between them

The single most common exam mistake is blurring these three. Lock the distinction now. Tap a card to flip it.

A horizontal bar chart of every task against time. Shows **sequencing, durations, dependencies, milestones** and the critical path. It plans and monitors the *schedule*. Forward-looking.

A dated log of ideas generated, choices made, problems hit and modifications applied — **and the reasoning behind them**. It records the *story*. Backward-looking.

Keeping numbered/dated copies of the work-in-progress so you can **roll back**, compare, and prove authorship. It protects the *files*. Reversible.

Anatomy of a Gantt chart

A Gantt chart is a bar chart on its side: each task is a horizontal bar whose **position** is its start, and whose **length** is its duration. Six features carry marks — learn every one.

The individual jobs that make up the project, listed down the left (e.g. “Write GPS code”, “Print enclosure”, “ User testing ”).

How long each task is expected to take — shown by the bar's length along the time axis (days/weeks across the top).

The order tasks run in. Some run in parallel (overlapping bars); some must run one after another.

An arrow/link meaning “this task can't start until that one finishes.” Testing *depends on* a printed enclosure existing.

A key checkpoint with zero duration, drawn as a diamond ♦ — e.g. “Prototype complete”, “Submission”. Markers you steer toward.

The chain of dependent tasks that takes the **longest** to complete. It sets the minimum project length. If any task *on* it slips, the whole project finishes late. Tasks *off* it have **slack** (float) — they can slip a little without hurting the deadline.

It tells the project manager **exactly which tasks to protect**. Jade doesn't have to micromanage everyone equally — she watches the critical-path tasks like a hawk, because those are the only ones that can blow the deadline. The next lab lets you feel this directly.

Critical-path explorer

FloatMate's build as a live 3D Gantt chart. Each bar is a task; arrows are dependencies; the diamond is the “Prototype complete” milestone. **Drag a task to make it slip** and watch what happens to the milestone — then learn why some slips hurt and some don't.

A Gantt chart you never update is useless

Drawing the chart in Week 1 is *planning*. The marks are in **monitoring** — comparing planned vs actual every week and **annotating** the changes. Examiners love the three-column change record.

What you altered on the plan — e.g. “extended testing by 3 days.”

Why it was necessary — e.g. “GPS drifted indoors; needed outdoor retests.”

Effect on the schedule — e.g. “pushed submission rehearsal to Wk 9; documentation task shortened.”

Pick what happened on Jade's project and generate the annotation an assessor wants to see.

Project journals & logs

A **project journal** (often kept in OneNote, a Google Doc or a shared wiki) is a running, **dated** record of the build. A Gantt chart says *when*; the journal says *what happened and why*.

What goes in a strong entry

Date · who did what · ideas generated · decisions made and the **reasoning** · problems hit and how they were solved · modifications to the solution · next steps. Evidence of thinking, not just activity.

Why it earns marks

It captures **design decisions** the final product can't show, supports **delegation** (everyone sees the latest state), and is core **authentication** evidence that the work developed over time.

A Gantt chart is the **recipe's timing** (“simmer 20 min, then add the rice”). The journal is the **cook's notes in the margin** (“used less chilli this time — last batch was too hot for the family”). Both matter; only the second explains your *choices*.

Version control

Version control means deliberately keeping **separate dated copies** of the work as it develops — so a later mistake never destroys earlier good work, and you can always show how the solution evolved.

□ Naming convention

A consistent, sortable pattern: floatmate_v0.3_2026-05-08.py. Include a version number and date; flag the last known-good copy as _working. Never just “final”, “final2”, “realfinal”.

▲ Where copies live

Keep copies in more than one place — a network drive, an external drive, or secure cloud storage — so one failure can't wipe the project. Backups and versioning work together.

Rollback: a broken change can be undone by returning to the last working version.

Collaboration: the team always knows which copy is current, avoiding two people overwriting each other.

Authentication: a trail of dated versions evidences genuine, progressive development.

Professional teams use tools like **Git/GitHub** that automate this, but the examinable idea is the *principle*: keep recoverable, dated copies of work in progress.

Version time-machine

Each glowing slab is a saved version of FloatMate's code, stacked oldest-to-newest. Version **v0.4** introduced a bug that broke GPS. Use the controls to **roll back** to the last working version — exactly what version control buys you.

How the three tools drive delegation & monitoring

The dot point ends with the purpose: **for delegation and monitoring**. Here's the full loop on Jade's team.

The weekly loop

1 • Delegate (Gantt): the chart's parallel, non-dependent tasks are split across the four members by their strengths — Sam (code), Priya (3D/print), Noah (testing), Jade (coordination + journal).

2 • Do & record (Journal + versions): as each person works, they log decisions in the journal and save dated versions of their files.

3 • Monitor (Gantt + journal): every Friday the team compares planned vs actual, annotates slippage (change/reason/impact), and re-checks the critical path.

4 • Recover when needed (versions): a broken build rolls back to the last working copy, and the loop continues.

Examiners often ask about **coordination via roles and responsibilities**. The point: clear ownership (who is responsible for which task) plus shared visibility (everyone can see the plan, journal and current versions) is what turns four individuals into one functioning team.

Command-word decoder

The verb in a question tells you how much to write. Tap one.

P-E-L writing trainer

Prompt: Jade's team is building FloatMate over a 10-week term. *Explain* how a Gantt chart would help them coordinate and monitor the project. (3 marks)

VCE-style practice questions

Each question has a deliberately **weak** answer, a **strong** answer, and a marking guide. Attempt it first, reveal the models, then self-mark — your scores fill the dock.

Total self-marked: **0** / 17 — keep the dock open (bottom-right) to track your running tally.

Common traps & misconceptions

Exam trap

⚠ Confusing a Gantt chart with a project journal “The Gantt chart records the team's decisions and reasons.” Fix: the Gantt chart schedules tasks/time; the journal records decisions and reasoning.

Exam trap

⚠ Defining critical path as “the most important tasks” It isn't about importance. Fix: it's the longest chain of dependent tasks — the sequence that sets the minimum project duration.

Exam trap

⚠ Treating the Gantt chart as a one-off Drawing it once = planning only. Fix: stress it's a living document — monitored and annotated (change/reason/impact) throughout.

Exam trap

⚠ Forgetting milestones have zero duration A milestone is not a task. Fix: it's a checkpoint marker (◆) — “prototype complete”, “submission” — with no length.

Exam trap

⚠ Saying version control = “saving your file” Too thin for full marks. Fix: it's keeping multiple dated, named copies so you can roll back, compare, collaborate and authenticate.

Exam trap

⚠ Listing tools without stating purpose Naming “Gantt, journal, version control” alone is a low-band answer. Fix: always tie each tool to delegation, monitoring, recovery or authentication.

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. List · 1 mark

: correct description of a project journal (dated record of decisions/ideas/modifications). : correct description of version control (keeping recoverable dated copies of files) — must show the difference, not just list both.

✗ A weak answer

A project journal and version control are both ways of keeping track of your project so you don't lose your work. No distinction is actually drawn — it just says both 'track the project'. Zero clear difference = low band.

✓ A strong answer

A project journal is a dated log of the decisions, ideas and modifications made during development (the *why*). Version control is the practice of keeping multiple dated, named copies of the work files so earlier versions can be recovered (the *which copy*). The journal records reasoning; version control protects and recovers files.

Q2. Practice question · 1 mark

: critical path = longest sequence of dependent tasks / sets minimum project duration. : slipping a task on it delays the whole project (tasks off it have slack). : links to the manager — shows which tasks to prioritise/monitor for FloatMate.

✗ A weak answer

The critical path is the most important tasks in the project that you have to get done or it won't work. 'Most important tasks' is the classic misconception. It also never explains the manager benefit.

✓ A strong answer

The critical path is the longest chain of dependent tasks, which sets the minimum time the whole project can take. If any task on it slips, the project finish date slips too. It matters to Jade because it tells her exactly which tasks (e.g. print enclosure assemble test) to protect and monitor closely, since tasks off the path have slack and can absorb small delays without affecting submission.

Q3. Practice question · 1 mark

: records the change (extend the print task). : records the reason (failed/warped print). : records the impact on schedule (dependent tasks/milestone/finish move). : recognises it affects the critical path / mentions journal annotation or protective action.

✗ A weak answer

They should just change the dates on the Gantt chart and tell everyone it's going to take longer now. Generic — no change/reason/impact structure, no mention of the critical path or dependent tasks.

✓ A strong answer

The team should annotate the Gantt chart with a change record: the **change made** (extend the print task by one week), the **reason** (first print warped and failed the quality check) and the **impact** (because printing is on the critical path, the dependent assemble and test tasks shift later, moving the prototype milestone and finish back by a week). They would also note the change in the project journal and may reduce slack elsewhere or trim scope to protect the submission deadline.

Q4. Practice question · 1 mark

: delegation — plan reveals independent/parallel tasks to split. : delegation — clear ownership/roles avoid duplication or idle members. : authentication — dated journal/versions show progressive development. : authentication — this evidences the work is the team's own.

✗ A weak answer

Documenting helps everyone know what to do and proves you did the work yourself. Restates the question without explaining the mechanism for either part.

✓ A strong answer

(a) Delegation: a Gantt chart shows which tasks are independent and can run in parallel, so Jade can assign them to different members by strength (Sam-code, Priya-enclosure) with clear ownership and no duplicated effort. (b) Authentication: a dated project journal and a trail of dated versions show the solution developed progressively over the term, providing evidence to the teacher that the work is genuinely the team's own and not copied or produced in one sitting.